

Reflection TS NB comment resolutions: summary and rationale

Document #: P1390R1

Date: 2019-02-22

Project: Programming Language C++
SG7/Evolution/LEWG/LWG/Core

Reply-to: Matúš Chochlík
<chochlik@gmail.com>
Axel Naumann
<Axel.Naumann@cern.ch>
David Sankel
<dsankel@bloomberg.net>

Contents

- [1 Introduction](#)
- [2 Suggested core resolutions \(EWG / CWG\)](#)
 - [2.1 CH 004 \(04.3\) - SG10/CWG](#)
 - [2.2 CH 005 \(06p1\) - CWG](#)
 - [2.3 CH 006 \(06p1\) - CWG](#)
 - [2.4 CA 008 \(08\) - CWG](#)
 - [2.5 PL 009 \(10.1.7.2, 10.1.7.6\) - CWG](#)
 - [2.6 CH 010 \(10.1.7.2\) - SG7/EWG](#)
 - [2.7 CA 011 \(10.1.7.2\) - CWG](#)
 - [2.8 CH 013 \(10.1.7.6\) - CWG](#)
 - [2.9 CA 016 \(10.1.7.6 p1\) - CWG](#)
 - [2.10 CA 017 \(10.1.7.6 p1\) - CWG](#)
 - [2.11 CA 019 \(10.1.7.6 p1\) - CWG](#)
 - [2.12 CA 021 \(10.1.7.6 p1\) - CWG](#)
 - [2.13 US 024 \(10.1.7.6 p1\) - CWG](#)
 - [2.14 US 025 \(10.1.7.6 p1\) - CWG](#)
 - [2.15 CH 026 \(17.7.2.1 p09.10\) - CWG](#)
 - [2.16 CA 027 \(17.7.2.1 p1\) - CWG](#)
 - [2.17 CH 029 \(21.12\) - CWG](#)
 - [2.18 CA 030 \(21.12\) - CWG](#)
 - [2.19 CA 031 \(21.12\) - SG7/EWG](#)
 - [2.20 CH 032 \(21.12.2\) - CWG](#)
 - [2.21 PL 035 \(21.12.2\) - CWG](#)
 - [2.22 PL 036 \(21.12.2\) - SG7/EWG](#)
 - [2.23 CH 039 \(21.12.2, 21.12.3.9\) - CWG](#)
 - [2.24 CH 040 \(21.12.3\) - SG7/EWG](#)
 - [2.25 CH 041 \(21.12.3.10\) - SG7/EWG](#)
 - [2.26 CH 042 \(21.12.3.18\) - SG7/EWG](#)
 - [2.27 CH 043 \(21.12.3.6\) - CWG](#)
 - [2.28 CH 044 \(21.12.3.7\) - CWG](#)
 - [2.29 CA 045 \(21.12.4\) - CWG](#)
 - [2.30 CA 046 \(21.12.4\) - CWG](#)
 - [2.31 PL 047 \(21.12.4\) - SG7/EWG](#)

- [2.32 CA 049 \(21.12.4.1\) - CWG](#)
- [2.33 CH 050 \(21.12.4.10 p1\) - CWG](#)
- [2.34 CA 052 \(21.12.4.16 p1\) - CWG](#)
- [2.35 CA 053 \(21.12.4.16\) - SG7/EWG](#)
- [2.36 PL 054 \(21.12.4.18\) - SG7/EWG](#)
- [2.37 CH 057 \(21.12.4.3 p2.1\) - SG7/EWG](#)
- [2.38 CH 058 \(21.12.4.3\) - SG7/EWG](#)
- [2.39 CA 060 \(21.12.4.3 p2\) - CWG](#)
- [2.40 CA 061 \(21.12.4.3 p2\) - CWG](#)
- [2.41 CA 064 \(21.12.4.3 p2\) - CWG](#)
- [2.42 CA 065 and CA 066 \(21.12.4.3 p2\) - CWG](#)
- [2.43 CA 068 \(21.12.4.3 p2\) - CWG](#)
- [2.44 CH 072 \(21.12.4.5\) - SG7/EWG](#)
- [2.45 CH 074 \(21.12.4.6\) - SG7/EWG](#)
- [2.46 US 075 \(21.12.4.6\) - CWG](#)
- [2.47 CH 078 \(21.12.4.7 and many others\) - CWG](#)
- [2.48 CH 079 \(21.12.4.9\) - SG7/EWG](#)
- [2.49 CH 080 \(21.12.4.9\) - SG7/EWG](#)
- [2.50 CH 082 \(Annex C\) - CWG](#)
- [2.51 GB 083 \(Annex C\) - CWG](#)
- [2.52 GB 084 \(Classes\) - CWG](#)
- [2.53 CA 085 \(General\) - CWG!](#)
- [2.54 GB 086 \(General, 04.2\) - SG10/CWG](#)
- [3 Suggested library resolutions \(LEWG/LWG\)](#)
 - [3.1 CA 028 \(20.5.1\) - LEWG](#)
 - [3.2 CH 056 \(21.12.4.2\) - SG7/LWG](#)
- [4 Suggested editorial resolutions](#)
 - [4.1 CA 069](#)
 - [4.2 CA 070](#)
 - [4.3 CA 071](#)
 - [4.4 CA 076](#)
- [5 Acknowledgements](#)
- [6 References](#)

1 Introduction

This paper contains proposed resolutions to national body comments issued in [[N5325](#)] which summarizes responses to [[N4766](#)], the proposed Reflection TS.

2 Suggested core resolutions (EWG / CWG)

2.1 CH 004 (04.3) - SG10/CWG

See GB 086.

2.2 CH 005 (06p1) - CWG

Reject, an alias-declaration is not a namespace-alias.

2.3 CH 006 (06p1) - CWG

Recommend to reject the proposed change. The term *alias* can also refer to variable aliases.

Clarify by modifying the example in 21.12.3.4:

```
namespace N {
    struct A;
}
namespace M {
    using X = N::A;
}
+ struct B {
+     int i;
+ };
+ struct C {
+     using B::i;
+ };
    using M_X_t = reflexpr(M::X);
    using M_X_scope_t = get_scope_t<M_X_t>;
+ using C_i_t = reflexpr(C::i);
+ using C_i_scope_t = get_scope_t<C_i_t>;
```

The scope reflected by `M_X_scope_t` is `M`, not `N`; the scope reflected by `C_i_scope_t` is `C`, not `B`.

2.4 CA 008 (08) - CWG

Recommend to accept the proposed change with modification.

In 8.6, insert a new bullet 8.5, adjusting the subsequent bullet numbers:

(8) An expression is potentially constant evaluated if it is: ... `:::add` (8.5) — a *reflexpr-operand*, `:::`

Modify 10.1.7.6 as follows:

Any other *reflexpr-operand* renders the program ill-formed.

If the *reflexpr-operand* of the form *id-expression* is a constant expression, the type specified by the *reflexpr-specifier* also satisfies `reflect::Constant`.

(Note that the example provided as part of CA 008 is invalid as `reflexpr(foo(a))` specifies a type that satisfies `FunctionCallExpression`, not `Constant`.)

2.5 PL 009 (10.1.7.2, 10.1.7.6) - CWG

Recommend to accept the proposed change with modifications. Modify

Modify the new final paragraph of 10.1.7.2 as follows:

For a *reflexpr-operand* `x`, the type denoted by `reflexpr(x)` is ~~an implementation-defined~~ a type that satisfies the constraints laid out in 10.1.7.6.

Modify 10.1.7.6 as follows:

(1) ... A meta-object type is an unnamed, incomplete namespace-scope class type (Clause 12).

~~... The type specified by the `reflexpr-specifier` is implementation-defined.~~

2.6 CH 010 (10.1.7.2) - SG7/EWG

Propose to accept; this grammar is now allowed as part of *id-expression*, see CA 011.

2.7 CA 011 (10.1.7.2) - CWG

Recommend to accept the proposed change.

Modify 10.1.7.2 as follows:

```
reflexpr-operand:  
  ::  
  type-id  
—nested-name-specifieropt identifier  
—nested-name-specifieropt simple-template-id  
  nested-name-specifieropt namespace-name  
  id-expression
```

Modify 10.1.7.6 as follows:

~~For an operand that is a parenthesized expression (8.4.3), the type satisfies `reflect::ParenthesizedExpression`. For a parenthesized expression (*E*), whether or not itself nested inside a parenthesized expression, the expression *E* shall be either a parenthesized expression, a function-call-expression or a functional-type-conv-expression; otherwise the program is ill-formed.~~

~~For an operand of the form function-call-expression, the type satisfies `reflect::FunctionCallExpression`. If the postfix-expression of the function-call-expression is of class type, the function call shall not resolve to a surrogate call function (16.3.1.1.2). Otherwise, the postfix-expression shall name a function that is the unique result of overload resolution.~~

~~For an operand of the form functional-type-conv-expression (8.5.1.3), the type satisfies `reflect::FunctionalTypeConversion`. [Note: The usual disambiguation between function-style cast and a type-id (11.2) applies. [Example: The default constructor of class *x* can be reflected on as `reflexpr((X()))`, while `reflexpr(X())` reflects the type of a function returning *x*. —end example] —end note]~~

~~For an operand of the form identifier where identifier is a template type-parameter, the type satisfies both `reflect::Type` and `reflect::Alias`.~~

~~The identifier or simple-template-id is looked up using the rules for name lookup (6.4): if a nested-name-specifier is included in the operand, qualified lookup (6.4.3) of nested-name-specifier identifier or nested-name-specifier simple-template-id will be performed, otherwise unqualified lookup (6.4.1) of identifier or simple-template-id will be performed. The type specified by the reflexpr-specifier satisfies concepts depending on the result of the name lookup, as shown in Table 12.~~

~~If the reflexpr-operand designates a type-id not explicitly mentioned in Table 12, the type represented by the reflexpr-specifier satisfies `reflect::Type`. Any other reflexpr-operand renders the program ill-formed.~~

Modify the caption of Table 12 as follows:

reflect concept (21.12.3) that the type specified by a *reflexpr-specifier* satisfies, for a given *reflexpr-operand* ~~identifier or simple-template-id~~.

Replace Table 12 by the following table:

Category	<i>reflexpr-operand</i>	reflect Concept
Type	<i>class-name</i> designating a union	reflect::Record
	<i>class-name</i> designating a closure type	reflect::Lambda
	<i>class-name</i> designating a non-union class	reflect::Class
	<i>enum-name</i>	reflect::Enum
	template <i>type-parameter</i>	both reflect::Type and reflect::Alias
	<i>type-name</i> introduced by a <i>using-declaration</i>	both reflect::Type and reflect::Alias
	any other <i>typedef-name</i>	both reflect::Type and reflect::Alias
	any other <i>type-id</i>	reflect::Type
Namespace	<i>namespace-alias</i>	both reflect::Namespace and reflect::Alias
	any other <i>namespace-name</i>	reflect::Namespace
Expression	the name of a data member	reflect::Variable
	the name of a variable or structured binding	reflect::Variable
	the name of an enumerator	reflect::Enumerator
	the name of a function parameter	reflect::FunctionParameter
	the name of a captured entity	reflect::LambdaCapture
	parenthesized expression; see Note A, below	reflect::ParenthesizedExpression
	<i>function-call-expression</i> ; see Note B, below	reflect::FunctionCallExpression
	<i>functional-type-conv-expression</i> ; see Note C, below	reflect::FunctionalTypeConversion

Add the following new paragraph after the table:

Note A) For a parenthesized expression (E), whether or not itself nested inside a parenthesized expression, the expression E shall be either a parenthesized expression, a *function-call-expression*, or a *functional-type-conv-expression*; otherwise the program is ill-formed.

Note B) If the *postfix-expression* of the *function-call-expression* is of class type, the function call shall not resolve to a surrogate call function (16.3.1.1.2). Otherwise, the *postfix-expression* shall name a function that is the unique result of overload resolution.

Note C) [*Note: The usual disambiguation between function-style cast and a *type-id* (11.2) applies. [Example: The default constructor of class X can be reflected on as `reflexpr((X(.))`), while `reflexpr(X(.))` reflects the type of a function returning X. — end example] — end note]*

2.8 CH 013 (10.1.7.6) - CWG

Recommend to accept the proposed change.

Modify 10.1.7.6 as follows:

If the *reflexpr-operand* designates an entity or alias at block scope (6.3.3) or function ~~prototype~~parameter scope (6.3.4) and...

2.9 CA 016 (10.1.7.6 p1) - CWG

Recommend to accept the proposed change; Constant *reflexpr-operands* are now explicitly called out by the recommended modifications in CA 011. Also modify 21.12.3.17 as follows:

Constant is true if and only if T reflects ~~a constant expression (8.6)~~ an enumerator or a constexpr variable.

2.10 CA 017 (10.1.7.6 p1) - CWG

Recommend to accept the proposed change. Similar to types represented by the `decltype` specifier, fundamental and cv-qualified types must not have a scope either. Only declarations should be `ScopeMembers`.

Modify Table 12 as follows:

<u><i>decltype-specifier</i></u>	<u>both <code>reflect::Type</code> and <code>reflect::Alias</code></u>
<i>type-name</i> introduced by a <i>using-declaration</i>	<code>reflect::Type</code> and , <code>reflect::Alias</code> , <u>and <code>reflect::ScopeMember</code></u>

Modify 21.12.2 as follows:

```
- template <class T> concept Enum;      // refines Type and Scope
- template <class T> concept Record;   // refines Type and Scope
+ template <class T> concept Enum;     // refines Type, Scope, and ScopeMember
+ template <class T> concept Record;   // refines Type, Scope, and ScopeMember
  template <class T> concept Scope;    // refines Object
- template <class T> concept Type;     // refines Named and ScopeMember
+ template <class T> concept Type;     // refines Named
```

Modify 21.12.3.16 as follows:

```
- template <class T> concept Type = Named<T> && ScopeMember<T> && see below;
+ template <class T> concept Type = Named<T> && see below;
```

Add a note at the end of 21.12.3.16:

[Note: Some types T also satisfy `ScopeMember`; others, for instance those reflecting cv-qualified types or fundamental types, do not. — end note]

2.11 CA 019 (10.1.7.6 p1) - CWG

Recommend to accept the proposed change. Modify 10.1.7.6 as follows:

~~For a parenthesized expression (E), whether or not itself nested inside a parenthesized expression, the expression E shall be either a parenthesized expression, a function-call-expression or a functional-type-conv-expression; otherwise the program is ill-formed.~~

For a *reflexpr-operand* that is a parenthesized expression (E), E shall be a function-call-expression, functional-type-conv-expression, or an expression (E') that satisfies the requirements for being a *reflexpr-operand*.

2.12 CA 021 (10.1.7.6 p1) - CWG

Recommend to accept the proposed change with modification. Modify 10.1.7.6 as follows:

An entity or alias $\overline{AB}$ is *reflection-related* to an entity or alias $\overline{BA}$ if

- A and B are the same entity or alias,
- A is a variable or enumerator and B is the type of A,
- A is an enumeration and B is the underlying type of A,
- A is a class and B is a member or base class of A,
- A is a non-template alias that designates the entity B, ~~—A is a class nested in B (12.2.5);~~
- A is not the global namespace and B is an enclosing class or namespace of A,
- $\overline{BA}$ is the parenthesized expression ($\overline{AB}$),
- A is a lambda capture of the closure type B,
- A is the closure type of the lambda capture B,
- $\overline{AB}$ is the type specified by the *functional-type-conv-expression* $\overline{BA}$,
- $\overline{AB}$ is the function selected by overload resolution for a *function-call-expression* $\overline{BA}$,
- $\overline{AB}$ is the return type, parameter type, or function type of the function $\overline{BA}$, or
- $\overline{AB}$ is reflection-related to an entity or alias x and x is reflection-related to $\overline{BA}$.

Note: the modification to the suggested change consists in the clarification of enclosing classes, and the removal of the superfluous nested class case. The latter is subsumed by “member or base class of A”. The resolution was also modified to be such that A yields B, with a corresponding change in the position of A and B in the definition of the relation.

2.13 US 024 (10.1.7.6 p1) - CWG

Recommend to accept with modification. The intent is to disallow the following:

```
int x;
void f() {
    extern int x;
    reflexpr(x); // ERROR
}
```

See the modification in US 025.

2.14 US 025 (10.1.7.6 p1) - CWG

Recommend to accept proposal with modification:

Modify 10.1.7.6 as follows (assuming acceptance of the proposed resolution of US 024):

If the *reflexpr-operand* designates ~~an entity or alias declared at~~ a name whose declaration is enclosed in a block scope (6.3.3) or function prototype scope (6.3.4) and the named entity is neither captured nor a function parameter, the program is ill-formed.

2.15 CH 026 (17.7.2.1 p09.10) - CWG

Recommend to accept to proposed modification of 17.7.2.1:

- denoted by `reflexpr(operand)`, where *operand* designates a dependent type or a member of an unknown specialization or a value-dependent constant expression.

2.16 CA 027 (17.7.2.1 p1) - CWG

Recommend to accept the proposed modification of 17.7.2.1.

Add a new bullet after 9.9:

- denoted by `reflexpr(operand)`, where *operand* is a type-dependent expression or a (possibly parenthesized) *functional-type-conv-expression* with at least one type-dependent immediate subexpression, or

2.17 CH 029 (21.12) - CWG

Recommend to reject the proposed modification, as `inline` is not needed (has no actual effect).

2.18 CA 030 (21.12) - CWG

Recommend to accept the proposed change.

Modify 21.12.3.6 as follows:

```
- template <class T> concept Enumerator = Typed<T> && ScopeMember<T> && see below;  
+ template <class T> concept Enumerator = Constant<T> && see below;
```

Modify 21.12.3.7 as follows:

```
- template <class T> concept Variable = Typed<T> && see below;  
+ template <class T> concept Variable = Typed<T> && ScopeMember<T> && see below;
```

Modify 21.12.3.9 as follows:

```
- template <class T> concept Typed = Named<T> && see below;  
+ template <class T> concept Typed = Object<T> && see below;
```

Modify 21.12.3.10 as follows:

```
- template <class T> concept Namespace = Scope<T> && see below;  
+ template <class T> concept Namespace = Named<T> && Scope<T> && see below;
```

Modify 21.12.3.17 as follows:

```
- template <class T> concept Constant = ScopeMember<T> && Typed<T> && see below;  
+ template <class T> concept Constant = Typed<T> && ScopeMember<T> && see below;
```

Modify 21.12.3.20 as follows (for consistency's sake):

```
- template <class T> concept Callable = ScopeMember<T> && Scope<T> && see below;  
+ template <class T> concept Callable = Scope<T> && ScopeMember<T> && see below;
```

Modify 21.12.3.25 as follows (for consistency's sake):

```
- template <class T> concept Function = Callable<T> && Typed<T> && see below;  
+ template <class T> concept Function = Typed<T> && Callable<T> && see below;
```

Modify 21.12.3.31 as follows (for consistency's sake):

- `template <class T> concept ConversionOperator = Operator<T> && MemberFunction<T> && see below;`
+ `template <class T> concept ConversionOperator = MemberFunction<T> && Operator<T> && see below;`

2.19 CA 031 (21.12) - SG7/EWG

Recommend to accept the proposed change with modifications.

Modify 21.12.3.4 as follows:

(2) ~~Except for the type represented by the `constexpr` operator, Alias properties resulting from type transformations (21.12.4) are not retained.~~ Type transformations (21.12.4) never yield an Alias; instead, they yield the aliased entity.

At the end of the paragraph 2 of 21.12.4, add

Alias entities are not returned by meta-object operations (21.12.3.4).

Modify 21.12.4.7 as follows:

(1) A specialization of any of these templates with a meta-object type that is reflecting an incomplete type renders the program ill-formed. Such errors are not in the immediate context (17.9.2). Members introduced by *using-declarations* (10.3.3) are included in the sequences below where applicable; the scope of these members remains that of the declaration of the referenced entity. [Note: These members are not Aliases, see 21.12.4. A member injected into a derived class may have different access. — end note]

(2) The nested type named `type` is an alias to an `ObjectSequence` specialized with `RecordMember` types that reflect the following subset of direct non-template members of the class reflected by `T`:

2.20 CH 032 (21.12.2) - CWG

Recommend to accept the proposed change with modification; variable templates have no immediate context which would allow them to be SFINAE'd on.

Modify 21.12.2 as follows:

```
template <class T>
+ requires RecordMember<T> || Base<T>
    constexpr auto is_public_v = is_public<T>::value;
template <class T>
+ requires RecordMember<T> || Base<T>
    constexpr auto is_protected_v = is_protected<T>::value;
template <class T>
+ requires RecordMember<T> || Base<T>
    constexpr auto is_private_v = is_private<T>::value;
template <class T>
+ requires Variable<T> || Callable<T>
    constexpr auto is_constexpr_v = is_constexpr<T>::value;
template <class T>
+ requires Variable<T> || MemberFunction<T>
    constexpr auto is_static_v = is_static<T>::value;
template <class T>
+ requires Class<T> || MemberFunction<T>
    constexpr auto is_final_v = is_final<T>::value;
```

```

template <class T>
+ requires Constructor<T> || ConversionOperator<T>
    constexpr auto is_explicit_v = is_explicit<T>::value;
template <class T>
+ requires Namespace<T> || Callable<T>
    constexpr auto is_inline_v = is_inline<T>::value;
template <class T>
+ requires Base<T> || MemberFunction<T> || Destructor<T>
    constexpr auto is_virtual_v = is_virtual<T>::value;
template <class T>
+ requires MemberFunction<T> || Destructor<T>
    constexpr auto is_pure_virtual_v = is_pure_virtual<T>::value;
template <class T>
+ requires Variable<T> || Function<T>
    constexpr auto get_pointer_v = get_pointer<T>::value;

```

2.21 PL 035 (21.12.2) - CWG

Recommend to reject. As 21.12.3.27 notes:

Some types that satisfy `Constructor` also satisfy `SpecialMemberFunction`.

The definition of `SpecialMemberFunction` (21.12.3.27) uses “special member function (Clause 15)” and thus clearly includes special member functions that are constructors.

2.22 PL 036 (21.12.2) - SG7/EWG

Recommend to reject the proposed change of renaming the concept `Object` to `Metaobject`:

The meta-nature of `std::reflect::Object` is stated by the namespace it is defined in. There is no need to repeat this meta-nature by prepending “Meta”.

2.23 CH 039 (21.12.2, 21.12.3.9) - CWG

See CA 030.

2.24 CH 040 (21.12.3) - SG7/EWG

Recommend to accept the proposed modification by adding the following note to 21.12.3:

[*Note: unlike `std::is_enum`, `std::reflect::is_enum` operates on meta-object types. — end note*]

2.25 CH 041 (21.12.3.10) - SG7/EWG

See CA 030.

2.26 CH 042 (21.12.3.18) - SG7/EWG

Suggest to reject.

This is a misunderstanding. A ‘base’ in this case has a class which is accessible with `get_class`. This class then has a name.

2.27 CH 043 (21.12.3.6) - CWG

See CA 030.

2.28 CH 044 (21.12.3.7) - CWG

See CA 030.

2.29 CA 045 (21.12.4) - CWG

See resolution to CA 071.

Note: the resolution here should be should also be applied to [\[P1208R3\]](#).

2.30 CA 046 (21.12.4) - CWG

Recommend to accept the proposed change, with modifications: the file name should not be represented as an NTMBS but NTBS since multi-byte has encoding implications.

Modify 21.12.4.3 as follows:

(2) All specializations of `get_name<T>` and `get_display_name<T>` shall meet the `UnaryTypeTrait` requirements (23.15.1) with a static data member named `value` of type `const char (&)[N]`, referencing a static, constant expression character array (~~NTBS~~ NTMBS) of length `N`, as if declared as `static constexpr char STR[N] = ...;`.

Note: the resolution here should be should also be applied to [\[P1208R3\]](#).

2.31 PL 047 (21.12.4) - SG7/EWG

It is unclear how (or if) such a thing can be implemented. At any rate, this is something that can be added in the future if we get implementation experience that suggests this is possible and desirable.

2.32 CA 049 (21.12.4.1) - CWG

Recommend to accept with modification:

In 21.12.4.1, apply the following changes:

(3) ... representing some offset from the start of the line (for `get_source_column<T>`) of ~~the most recent~~ a declaration of the entity or typedef described by `τ`.

(4) ... The value of the NTBS is the presumed name of the source file (19.8) of ~~the most recent~~ a declaration of the entity or typedef described by `τ`.

This matches US 024 forbidding block-scope declarations as *reflexpr-operands*.

2.33 CH 050 (21.12.4.10 p1) - CWG

Recommend to accept the proposed modification, by applying the following change to 21.12.4.10:

(1) ... The nested type named type ~~is an alias to~~ designates `reflexpr(X)`, where `X` is the base class (without retaining possible Alias properties, see 21.12.3.4) reflected by `T`.

Also apply the change

nested type named type that ~~is an alias to~~ designates

for all occurrences in the document.

2.34 CA 052 (21.12.4.16 p1) - CWG

`reflexpr((X(X())))` does not invoke the copy constructor, due to guaranteed copy elision. An alternative approach to access constructors through an expression is `reflexpr((X(declval<X>())))`. A more reliable approach is to retrieve a class's constructors, take the subset of special member functions, and find the copy constructor back.

We believe that no normative change is needed to address this NB comment. To clarify the intent, we recommend to accept the proposed change, adding the following note at the end of 21.12.4.16:

[Note: `get_constructor t<get_subexpression t<reflexpr((I(I(.))).>>` is ill-formed because the *functional-type-conv* expression does not perform overload resolution for a constructor (15.8.3). — end note]

NOTE TO THE EDITOR: this *Note* shall be dropped when rebasing on C++14.

2.35 CA 053 (21.12.4.16) - SG7/EWG

Recommend to accept the proposed change with modification.

Modify 21.12.4.16 as follows:

For a ~~type-conversion~~ functional-type-conv-expression reflected by `T`, ~~the nested type named type~~ let `S` be the type specified by the type conversion (8.5.1.3 [expr.type.conv]). If a constructor is used for the initialization of `S`, the type `get_constructor<T>::type` is the Constructor reflecting the ~~constructor of the type specified by the type conversion, as selected by overload resolution. The program is ill-formed if no such constructor exists; that constructor; otherwise,~~ `get_constructor<T>` has no member named `type`. [Note: For example, fundamental types (6.7.1) do not have constructors. — end note]

2.36 PL 054 (21.12.4.18) - SG7/EWG

Recommend to accept.

(1) All specializations of these templates shall meet the `UnaryTypeTrait` requirements (23.15.1). If their template parameter reflects a member function that is static (for `is_static`), const (for `is_const`), volatile (for `is_volatile`), declared with a *ref-qualifier* & (for `has_lvalueref_qualifier`) or && (for `has_rvalueref_qualifier`), implicitly or explicitly virtual (for `is_virtual`), pure virtual (for `is_pure_virtual`), or ~~marked with `override`~~ overrides a member function of a base class (for `is_override`) or final (for `is_final`), the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

2.37 CH 057 (21.12.4.3 p2.1) - SG7/EWG

Recommend to reject.

This is already the existing behavior and lambda objects are specifically called out. See Section 21.12.4.3 paragraph 2.5. Lambda objects are not unnamed entities.

2.38 CH 058 (21.12.4.3) - SG7/EWG

Recommend to accept, see the resolution to CA 071.

Template aliases are not supported by N4766.

2.39 CA 060 (21.12.4.3 p2) - CWG

Recommend to accept, by applying the modification shown in CA 071.

2.40 CA 061 (21.12.4.3 p2) - CWG

See CH 058.

2.41 CA 064 (21.12.4.3 p2) - CWG

Recommend to accept, by applying the following modification to 21.12.4.3:

(2.4.6) - for T reflecting ~~all other simple type specifiers~~ a cv-unqualified fundamental type other than `std::nullptr_t`, the name stated in the “Type” column of Table 9 in (10.1.7.2);

2.42 CA 065 and CA 066 (21.12.4.3 p2) - CWG

Recommend to accept CA 065 and CA 066, both with modifications.

See the resolution to CA 071.

2.43 CA 068 (21.12.4.3 p2) - CWG

See CH 058.

2.44 CH 072 (21.12.4.5) - SG7/EWG

Recommend to accept: struct and class are conceptually the same, and `std::reflect::is_struct` wrongly suggests the need of `std::is_struct`. To clearly distinguish the `std::is_class` type trait from these meta functions, we recommend to rename them as proposed in the NB comment.

Apply the following modification to 21.12.2:

```
- template <Type T> struct is_class;  
+ template <Class T> struct uses_class_key;  
- template <Type T> struct is_struct;  
+ template <Class T> struct uses_struct_key;  
  template <Type T> struct is_union;  
  template <Typed T>
```

```

    using get_type_t = typename get_type<T>::type;
template <Type T>
    using get_reflected_type_t = typename get_reflected_type<T>::type;
template <Type T>
    constexpr auto is_enum_v = is_enum<T>::value;
- template <Type T>
-     constexpr auto is_class_v = is_class<T>::value;
+ template <Class T>
+     constexpr auto uses_class_key_v = uses_class_key<T>::value;
- template <Type T>
-     constexpr auto is_struct_v = is_struct<T>::value;
+ template <Class T>
+     constexpr auto uses_struct_key_v = uses_struct_key<T>::value;

```

Apply the following modification to 21.12.4.5:

```

template <TypeClass T> struct is_classuses_class_key;

template <TypeClass T> struct is_structuses_struct_key;

```

(8) All specializations of is_class_uses_class_key<T> and is_struct_uses_struct_key<T> shall meet the UnaryTypeTrait requirements (23.15.1). If T reflects a class ~~with~~ for which all declarations use class-key class (for is_class_uses_class_key<T>) or struct (for is_struct_uses_struct_key<T>), the base characteristic of the respective template specialization is true_type, ~~otherwise it is false_type~~. If T reflects a class for which no declaration uses class-key class (for uses_class_key<T>) or struct (for uses_struct_key<T>), the base characteristic of the respective template specialization is false_type. Otherwise, it is unspecified whether the base characteristic is true_type or false_type. If the same class has redeclarations with both class-key class and class-key struct, the base characteristic of the template specialization of exactly one of is_class<T> and is_struct<T> can be true_type, the other template specialization is false_type; the actual choice of value is unspecified.

2.45 CH 074 (21.12.4.6) - SG7/EWG

To allow future extensions of template reflection, we recommend to accept this comment with modification.

To represent the scope of template parameters also for variable templates, a new concept needs to be introduced, serving as the scope of template parameters. For the time being, this concept has no operations (beyond that of Object) defined.

Modify 21.12.2 as follows:

```

template <class T> concept ObjectSequence; // refines Object
+ template <class T> concept TemplateParameterScope; // refines Scope
template <class T> concept Named; // refines Object

```

Add a new paragraph 21.12.3.3, moving the existing 21.12.3.3 to 21.12.3.4, etc:

21.12.3.3 Concept TemplateParameterScope

```

template <class T> concept TemplateParameterScope = Scope<T> && see below;

```

TemplateParameterScope<T> is true if and only if T is a Scope reflecting the scope of a template type-parameter, generated by a metaobject operation. [Note: It represents the template parameter scope (6.3.9), providing a scope to the Alias reflecting a template type-parameter. — end note].

Modify 21.12.3.4 as follows:

[Note: The Scope of an Alias is the scope that the alias was injected into. For an Alias reflecting a template type-parameter, that scope is its TemplateParameterScope. — end note]

Add the following sentence to the end of 21.12.4.6, paragraph 2:

(2) ... For a template type-parameter, this innermost scope is the TemplateParameterScope representing the template parameter scope in which it has been declared.

2.46 US 075 (21.12.4.6) - CWG

Recommend to accept the proposed change, by modifying 21.12.4.6 as follows:

(2) ... class scope, enumeration scope, function scope ~~(in the case where the function's parameters),~~
or ...

2.47 CH 078 (21.12.4.7 and many others) - CWG

Recommend to accept the proposed change.

Modify 21.12.4.7 as follows:

(2) ... The nested type named type ~~is an alias to an ObjectSequence specialized with RecordMember types that~~ designates a meta-object type satisfying ObjectSequence, containing elements which satisfy RecordMember and reflect the following subset of non-template members of the class reflected by T:

(5) ... The nested type named type ~~is an alias to an ObjectSequence specialized with RecordMember types that~~ designates a meta-object type satisfying ObjectSequence, containing elements which satisfy RecordMember and reflect the following subset of function members of the class reflected by T:

(10) ... The nested type named type ~~is an alias to an ObjectSequence specialized with Type types that~~ designates a meta-object type satisfying ObjectSequence, containing elements which satisfy Type and reflect the following subset of types declared in the class reflected by T:

(13) ... The nested type named type ~~is an alias to an ObjectSequence specialized with Base types that~~ designates a meta-object type satisfying ObjectSequence, containing elements which satisfy Base and reflect the following subset of base classes of the class reflected by T:

Modify 21.12.4.8 as follows:

(2) ... The nested type named type ~~is an alias to an ObjectSequence specialized with Enumerator types that~~ designates a meta-object type satisfying ObjectSequence, containing elements which satisfy Enumerator and reflect the enumerators of the enumeration type reflected by T. ...

Modify 21.12.4.13 as follows:

(1) ... The nested type named type ~~is an alias to an ObjectSequence specialized with FunctionParameter types that~~ designates a meta-object type satisfying ObjectSequence, containing elements which satisfy FunctionParameter and reflect the parameters of the function reflected by T. ...

Modify 21.12.4.23 as follows:

- (1) ... The nested type named type ~~is an alias to an ObjectSequence specialized with LambdaCapture types that~~ designates a meta-object type satisfying ObjectSequence, containing elements which satisfy LambdaCapture and reflect the captures of the closure object reflected by τ .
...

2.48 CH 079 (21.12.4.9) - SG7/EWG

The lack of an interface to determine thread-local storage duration is surprising; we recommend to accept the proposed modification.

Modify 21.12.2 as follows:

```
template <Constant T> struct get_constant;  
template <Variable T> struct is_constexpr<T>;  
template <Variable T> struct is_static<T>;  
+ template <Variable T> struct is_thread_local;  
template <Variable T> struct get_pointer<T>;  
  
template <Constant T>  
    constexpr auto get_constant_v = get_constant<T>::value;  
+ template <Variable T>  
+     constexpr auto is_thread_local_v = is_thread_local<T>::value;
```

Modify paragraph 4 of 21.12.4.9 as follows:

- (3) template <Variable T> struct is_static<T>;
(4) template <Variable T> struct is thread local;

All specializations of `is_static<T>` and `is thread local<T>` shall meet the `UnaryTypeTrait` requirements (23.15.1). If τ reflects a variable with static (for `is static`) or thread (for `is thread local`) storage duration, the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

2.49 CH 080 (21.12.4.9) - SG7/EWG

Recommend to accept the proposed change, by adding the following sentence at the beginning of paragraph 4:

- (4) ~~All specializations~~ If τ reflects a reference with static storage duration, and the reference has no prior initialization or has not been initialized with an object of static storage duration, the specialization of `get_pointer<T>` has no member named type. Otherwise, the specialization of `get_pointer<T>` shall meet the `UnaryTypeTrait` requirements (23.15.1),...

2.50 CH 082 (Annex C) - CWG

Recommend to accept the proposed change, by editing paragraph 1 in C.5.1 Clause 5: lexical conventions [diff.cpp17.lex]:

- (1) **Rationale:** Required for new features. The `requires` keyword is added to introduce constraints through a *requires-clause* or a *requires-expression*. The `concept` keyword is added to enable the definition of concepts (17.6.8). The `reflexpr` keyword is added to introduce meta-data through a *reflexpr-specifier*.

Effect on original feature: Valid ISO C++ 2017 code using `concept`~~or~~, `requires`, `or reflexpr` as an identifier is not valid in this International Standard.

2.51 GB 083 (Annex C) - CWG

See CH 082.

2.52 GB 084 (Classes) - CWG

See CH 013.

2.53 CA 085 (General) - CWG!

The authors believe that this comment should be addressed to CWG, rather than SG7/LEWG.

The authors recommend to add a new final sentence to the last paragraph of 10.1.7.6 as follows:

Reflecting directly or indirectly upon `std::align_val_t` [basic.stc.dynamic] without inclusion of the header `<new>` is ill-formed; no diagnostic required.

2.54 GB 086 (General, 04.2) - SG10/CWG

Propose to accept the suggested modification, by inserting a new 4.3 between 4.2 and 4.3:

4.3 Feature-testing recommendations [intro.features]

(1) An implementation that provides support for this Technical Specification shall define the feature test macros in Table 2.

<u>Macro name</u>	<u>Value</u>	<u>Header</u>
<u><code>__cpp_reflection</code></u>	<u>201902</u>	<u>none</u>
<u><code>__cpp_lib_reflection</code></u>	<u>201902</u>	<u><code><reflect></code></u>

Table 2 - Feature-test macro(s).

3 Suggested library resolutions (LEWG/LWG)

3.1 CA 028 (20.5.1) - LEWG

Suggestion to accept this request.

- Add `<experimental/reflect>` to Table 19 “C++ headers for freestanding implementations” referenced by 20.5.1.3.

3.2 CH 056 (21.12.4.2) - SG7/LWG

Modify definition of `unpack_sequence_t` from [reflect.synopsis] as follows.

```
template <template <class...> class Tpl, ObjectSequence T>
-   constexpr auto unpack_sequence_t = unpack_sequence<Tpl, T>::type;
+   using unpack_sequence_t = typename unpack_sequence<Tpl, T>::type;
```

4 Suggested editorial resolutions

4.1 CA 069

Suggest to reject; insufficient motivation for a change.

4.2 CA 070

Suggest to accept with modification.

See the resolution to CA 071.

4.3 CA 071

Suggest to accept the proposed change with modifications; this is a non-editorial change.

Replace the bulleted list of 21.12.4.3/2 by the following:

The value of `get_display_name_v<T>` is the empty string if T reflects an unnamed entity; otherwise the value is implementation defined.

The value of `get_name_v<T>` refers to a string literal whose *s-char-sequence* is obtained by the first matching case in the following list:

- for T reflecting an alias, the unqualified name of the aliasing declaration;
- for T reflecting an unnamed entity, the empty string.
- for T reflecting a specialization of a class, function (except for conversion functions, constructors, operator functions, or literal operators), or variable template, its *template-name*;
- for T reflecting a cv-unqualified type that is
 - a *type-parameter*, the identifier introduced by the *type-parameter*;
 - a class type, its *class-name*;
 - an enumeration type, its *enum-name*;
 - a fundamental type other than `std::nullptr_t`, the name stated in the “Type” column of Table 9 in (10.1.7.2);
- for T reflecting
 - a namespace, its *namespace-name*;
 - a variable, enumerator, data member, or function, its unqualified name;
 - a function parameter, its name;
 - a constructor, the *injected-class-name* of its class;
 - a destructor, the *injected-class-name* of its class, prefixed by the character ‘~’;
 - an operator function, the operator element of the relevant *operator-function-id*;
 - a literal operator, the *s-char-sequence* `\"\\\" followed by the literal suffix identifier of the operator’s literal-operator-id;`
 - a conversion function, the same characters as `get_name_v<R>`, with R reflecting the type represented by the *conversion-type-id*.
- In all other cases, the string’s value is the empty string.

4.4 CA 076

Suggest to reject. It is clear enough as is.

5 Acknowledgements

Special thanks to Michael Park who is responsible for the pandoc extensions [[PBLOG](#)] used to create this paper.

6 References

[N4766] Matúš Chochlík, Axel Naumann, and David Sankel. 2018. Working Draft, C++ Extensions for Reflection.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4766.pdf>

[N5325] Secretariat: ANSI (United States). 2018. Summary of Voting on PDTS 23619, Technical Specification – C++ Extensions for Reflection.

[P1208R3] Robert Douglas and Corentin Jabot. 2018. Source-Code Information Capture.

<https://wg21.link/P1208R3>

[PBLOG] Michael Park. 2018. How I format my C++ papers.

<https://mpark.github.io/programming/2018/11/16/how-i-format-my-cpp-papers/>